

Task 1: Create a neuron_output Function

Formula

$$\text{output} = (\text{input} \times \text{weight}) + \text{bias}$$

```
def neuron_output(input_value, weight, bias):  
    output = input_value * weight + bias  
    return output
```

Test

```
result = neuron_output(10, 0.8, 2)  
print("Output:", result)
```

Output

Output: 10.0

Task 2: Apply the Function to Instagram Likes

Suppose the likes on five Instagram posts are:

[50, 80, 120, 200, 300]

Weight = **0.8**

Bias = **2**

```
def neuron_output(input_value, weight, bias):  
    return input_value * weight + bias
```

```
likes = [50, 80, 120, 200, 300]
```

```
weight = 0.8
```

```
bias = 2
```

```
print("Neuron Outputs:\n")
```

```
for like in likes:
```

```
    output = neuron_output(like, weight, bias)  
    print("Likes:", like, "Output:", output)
```

Output

Neuron Outputs:

Likes: 50 Output: 42.0

Likes: 80 Output: 66.0
Likes: 120 Output: 98.0
Likes: 200 Output: 162.0
Likes: 300 Output: 242.0

Task 3: Initialize Neuron Weights

Three initialization methods:

- Zeros Initialization
- Random Initialization
- Xavier Initialization

```
import numpy as np
```

```
n = 10
```

```
# 1. Zero Initialization
```

```
zeros_weights = np.zeros(n)
```

```
# 2. Random Initialization (-1 to 1)
```

```
random_weights = np.random.uniform(-1, 1, n)
```

```
# 3. Xavier Initialization
```

```
xavier_weights = np.random.normal(0, 1/np.sqrt(n), n)
```

```
print("Zero Initialization:")
```

```
print(zeros_weights)
```

```
print("\nRandom Initialization:")
```

```
print(random_weights)
```

```
print("\nXavier Initialization:")
```

```
print(xavier_weights)
```

Sample Output

Zero Initialization:

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Random Initialization:

```
[-0.72  0.34 -0.44  0.81 -0.15 ...]
```

Xavier Initialization:

[0.09 -0.21 0.15 -0.04 0.31 ...]

Task 4: Importance of Bias

```
def neuron_output(input_value, weight, bias):  
    return input_value * weight + bias
```

```
input_value = 0
```

```
weight = 0.5
```

```
# Without Bias
```

```
output1 = neuron_output(input_value, weight, 0)
```

```
# With Bias
```

```
output2 = neuron_output(input_value, weight, 5)
```

```
print("Output with bias = 0 :", output1)
```

```
print("Output with bias = 5 :", output2)
```

```
# Explanation:
```

```
# Even when the input is 0,
```

```
# the neuron with bias = 5 produces an output of 5.
```

```
# This shows that the bias shifts the output and allows
```

```
# the neuron to activate even when the input is zero.
```

Output

```
Output with bias = 0 : 0.0
```

```
Output with bias = 5 : 5.0
```

Explanation

- Without bias:
 - $\text{Output} = (0 \times 0.5) + 0 = \mathbf{0}$
- With bias:
 - $\text{Output} = (0 \times 0.5) + 5 = \mathbf{5}$

Bias shifts the activation and makes the neuron more flexible in learning.

Task 5: Multiple Inputs and Weights (Without NumPy)

Formula:

$$\text{Output} = \sum(\text{input}_i \times \text{weight}_i) + \text{bias}$$

Inputs:

[2,4,1]

Weights:

[0.6,0.3,0.1]

Bias:

1

```
def neuron_output(inputs, weights, bias):
```

```
    weighted_sum = 0
```

```
    for i in range(len(inputs)):
```

```
        weighted_sum += inputs[i] * weights[i]
```

```
    return weighted_sum + bias
```

```
inputs = [2, 4, 1]
```

```
weights = [0.6, 0.3, 0.1]
```

```
bias = 1
```

```
result = neuron_output(inputs, weights, bias)
```

```
print("Weighted Sum + Bias =", result)
```

Output

Weighted Sum + Bias = 3.5

Manual Calculation

$$(2 \times 0.6) = 1.2$$

$$(4 \times 0.3) = 1.2$$

$$(1 \times 0.1) = 0.1$$

$$\begin{aligned} \text{Weighted Sum} &= 1.2 + 1.2 + 0.1 \\ &= 2.5 \end{aligned}$$

$$\begin{aligned}\text{Final Output} &= 2.5 + 1 \\ &= 3.5\end{aligned}$$